Clash Royale Vision System: An AI Agent for Merge Fight Tactics

Justin Bonner
CSC 598 - Computer Vision with Deep Learning

November 10, 2025

Abstract

This project develops an autonomous AI agent for playing Clash Royale's Merge Fight Tactics mode using computer vision and deep reinforcement learning. The system integrates real-time object detection using YOLOv11, game state analysis, and strategic decision-making through a hierarchical state machine. We have successfully implemented and trained a YOLOv11 model that achieves 86.2% mAP@50 for detecting 53 different game objects including troops, cards, and UI elements. The detection system forms the foundation for autonomous gameplay, enabling the bot to perceive game state in real-time. This report presents our progress on data collection pipelines, model training, and initial detection results, along with our plan to integrate reinforcement learning for strategic decision-making in the final implementation.

1 Introduction

Clash Royale's Merge Fight Tactics represents a challenging domain for AI agents due to its combination of strategic planning, resource management, and real-time decision-making. Unlike traditional video games where emulators provide direct access to game state, Merge Fight Tactics requires computer vision to extract information from visual frames, making it an ideal testbed for vision-based AI systems.

1.1 Game Mechanics

Merge Fight Tactics is a four-player, round-based strategy game with two distinct phases:

- **Deploy Phase**: Players purchase troops from a shop using elixir, merge duplicate units to create stronger versions ($\rightarrow$ $\rightarrow$ $\rightarrow$), and position troops strategically on the battlefield. Each troop has traits, and collecting multiple troops with matching traits activates bonus effects.
- **Battle Phase**: Troops automatically engage in combat. Players can continue purchasing and selling troops but cannot reposition them.

The game features a shared troop pool across all players, adding an additional layer of strategic depth as popular units become scarce. Critically, the turn-based nature allows for latency in model inference, making it feasible to run complex decision-making algorithms without real-time performance constraints.

1.2 Project Objectives

This project aims to create an end-to-end AI system capable of:

1. **Visual Perception**: Detecting and classifying game objects (troops, cards, resources) in real-time using deep learning
2. **State Understanding**: Parsing visual information into structured game state representations
3. **Strategic Decision-Making**: Using reinforcement learning to make optimal decisions about troop purchases, merges, and positioning
4. **Autonomous Gameplay**: Executing actions through screen interaction to play complete games

The modular architecture separates computer vision, game logic, and decision-making components, allowing independent development and testing of each subsystem.

2 Related Work

Our approach builds upon several key areas of research in computer vision and game-playing AI. YOLOv11 [**?**] provides state-of-the-art real-time object detection, which we apply to detecting game elements. Silva et al. [**?**] demonstrate combining YOLO with deep reinforcement learning for autonomous systems, providing a blueprint for our vision-based bot. Mnih et al. [**?**] show that deep Q-networks can learn to play games from visual input, though our approach uses explicit object detection rather than end-to-end pixel processing. Heinrich and Silver [**?**] develop RL agents for poker using self-play, a training paradigm we plan to adopt. Finally, Roboflow [**?**] provides the platform we use for dataset annotation and model training.

3 Methodology

Our system architecture consists of four main components: data collection, model training, object detection, and game state management.

3.1 Data Collection Pipeline

We developed an automated screenshot collection system (`screenshot_collector.py`) that captures gameplay images at randomized intervals (1-3 seconds) while playing Merge Fight Tactics. The collector implements several key features:

- **Session Management**: Organizes captures into timestamped sessions with metadata tracking
- **Region Specification**: Captures specific screen regions rather than full screenshots to focus on game-relevant areas
- **Multi-monitor Support**: Automatically detects and handles multiple display configurations
- **Randomized Intervals**: Varies capture timing to ensure diverse game states

Through multiple collection sessions, we gathered over 200 raw screenshots covering various game phases, troop configurations, and UI states. These images were then manually curated to select 111 high-quality examples for annotation.

3.2 Dataset Creation and Annotation

Using the Roboflow platform, we annotated 111 images with bounding boxes and class labels for 53 different object categories:

- **Deployed Troops** (21 classes): archer, baby_dragon, knight, pekka, wizard, etc.
- **Shop Cards** (31 classes): card_archer, card_knight, card_pekka, etc.
- **UI Elements** (1 class): skeleton indicator

The distinction between deployed troops and shop cards is critical for game state understanding—troops on the battlefield have different properties and available actions compared to cards in the shop.

The dataset was split into training (85 images, 76.6%), validation (18 images, 16.2%), and test (8 images, 7.2%) sets. Roboflow's preprocessing pipeline applied automatic orientation correction and resized images to 640×640 pixels with stretching to maintain consistent input dimensions for YOLO.

3.3 Model Training

We used the Roboflow platform to train our YOLOv11 nano object detection model. The annotated dataset of 111 images was uploaded to Roboflow, which handled the training infrastructure, augmentation, and model optimization. The model was trained for 100 epochs on the 53-class dataset, achieving strong detection performance on the validation set.

3.4 Computer Vision Architecture

The detection system processes screenshots through the following pipeline:

1. **Screenshot Capture**: Using the MSS library for efficient screen capture at 10-30 FPS
2. **Region Extraction**: Isolating game-relevant regions (battlefield, shop, resource counters)
3. **YOLO Inference**: Running YOLOv11 detection to identify and classify objects
4. **Post-processing**: Non-maximum suppression (NMS) to eliminate duplicate detections
5. **State Extraction**: Converting bounding boxes and class predictions into structured game state

Specialized detector modules handle different game regions:

- **BoardDetector**: Analyzes the 7×4 grid battlefield using computer vision to detect troop types, levels, and positions
- **GameOverDetector**: Uses OCR (EasyOCR) to identify victory/defeat states and UI buttons
- **ShopDetector** (planned): Will detect available shop cards and their costs
- **ResourceDetector** (planned): Will track elixir

3.5 Game State Representation

For now, we are storing game state as an array of troops that the player currently has deployed on the battlefield along with the current elixir count. We are still determining the best approach for representing more complex game state information such as shop contents, opponent states, and trait combinations. This simplified representation is sufficient for initial detection testing but will need to be expanded for strategic decision-making in future work.

3.6 Technology Stack

The system is implemented in Python 3.10+ with the following key dependencies:

- **Computer Vision**: Ultralytics (YOLOv11), OpenCV, Pillow
- **Deep Learning**: PyTorch, EasyOCR
- **Screen Interaction**: MSS (screenshot capture), PyAutoGUI (planned for action execution)
- **User Interface**: Textual (terminal UI), Rich (formatted output)
- **Data Management**: NumPy, Pandas

4 Results

4.1 Detection Performance

The YOLOv11 model trained on our custom dataset achieved strong performance on the validation set:

Table 1: YOLOv11 Detection Metrics on Validation Set

Metric	Value
mAP@50	86.2%
Precision	71.6%
Recall	77.8%

These results demonstrate that the model reliably detects and classifies game objects with high accuracy. The 86.2% mAP@50 indicates strong localization and classification performance across all 53 object classes. The precision-recall balance (71.6% and 77.8%) shows the model errs slightly toward false positives rather than missing objects—a favorable tradeoff for gameplay, as detecting extra objects is less detrimental than missing critical troops or cards.

4.2 Qualitative Results

The model successfully:

- Identifies multiple troop types simultaneously (archers, knights, wizards, etc.)
- Distinguishes between deployed troops and shop cards
- Handles varying scales, orientations, and lighting conditions
- Maintains accuracy in cluttered scenes with overlapping units

4.3 System Integration

Beyond model performance, we have successfully integrated the detection system into a working bot framework:

- **State Machine**: Implemented game phase detection and state transitions
- **Action System**: Developed modules for screen interaction (clicks, drags) to execute bot decisions
- **Real-time Processing**: Achieved detection latency under 50ms per frame on GPU, meeting real-time requirements

However, the strategic decision-making component remains rule-based and simplistic, forming the primary focus for future work.

5 Plan for Completion

To complete the project for the final report, we will focus on four key areas:

5.1 Reinforcement Learning Implementation

The current bot uses hard-coded rules for decision-making. We will implement a deep reinforcement learning agent to learn optimal strategies:

- **State Representation**: Design a feature vector encoding game state (troop compositions, resources, opponent status)
- **Action Space**: Define discrete actions (buy specific card, merge troops, position unit, end turn)
- **Reward Function**: Formulate rewards based on match outcome (win/loss), damage dealt/taken, and intermediate goals
- **Algorithm Selection**: Evaluate Proximal Policy Optimization (PPO) vs. Deep Q-Networks (DQN) for this domain
- **Training Infrastructure**: Implement self-play for rapid policy improvement

We will train the RL agent initially in a simulated environment before deploying to actual gameplay.

5.2 Expanded Training Dataset

To improve detection accuracy and handle edge cases:

- **Additional Data Collection**: Gather 300-500 more annotated images covering rare troops, unusual board states, and different screen resolutions
- **Augmentation**: Explore additional augmentation techniques (color jittering, noise injection) to improve robustness
- **Active Learning**: Identify and annotate images where the current model struggles
- **Iterative Training**: Retrain YOLOv11 with expanded dataset to push mAP above 90%

5.3 End-to-End Gameplay Bot

Integrate all components into a fully autonomous system:

- **Complete Detection Suite**: Implement missing detectors (shop prices, elixir counter, opponent boards)

- **Strategic Planning**: Connect RL agent to detection outputs and action execution
- **Game Flow Management**: Handle menu navigation, game start/end, error recovery
- **Multi-game Sessions**: Enable the bot to play consecutive games for extended testing

5.4 Performance Evaluation

Rigorously assess bot performance through quantitative and qualitative metrics:

- **Win Rate Analysis**: Play 50-100 games and measure win percentage against random opponents
- **Decision Quality**: Analyze replays to evaluate strategic choices (merge efficiency, trait utilization, resource management)
- **Comparison Baseline**: Compare RL-based decisions to rule-based strategies
- **Failure Case Analysis**: Identify scenarios where the bot performs poorly and iterate on training

6 Conclusion

We have successfully developed the computer vision foundation for an autonomous Clash Royale Merge Fight Tactics bot. The YOLOv11 model achieves 86.2% mAP@50 for detecting 53 game object classes, demonstrating robust real-time visual perception. The modular architecture separates detection, state management, and decision-making, enabling independent development and testing.

The primary remaining challenge is implementing intelligent strategic decision-making through reinforcement learning. With the vision system proven effective, we can focus the final phase on training an RL agent to master game strategy through self-play. This work demonstrates the feasibility of vision-based AI agents for complex strategy games and provides a framework applicable to other visually-driven gaming domains.

7 References

1. **YOLOv11 Architecture**
   `https://arxiv.org/pdf/2410.17725`
2. **Combining YOLO and Deep Reinforcement Learning for Autonomous Driving**
   `https://www.scitepress.org/Papers/2022/109136/109136.pdf`
3. **Playing Atari Games with Deep Reinforcement Learning and Human Checkpoint Replay**
   `https://www.researchgate.net/publication/305401668_Playing_Atari_Games_with_Deep_Reinforcement_Learning_and_Human_Checkpoint_Replay`

4. **Deep Reinforcement Learning from Self-Play in No-limit Texas Hold'em Poker**
https://www.researchgate.net/publication/357245384_Deep_Reinforcement_Learning_from_Self-Play_in_No-limit_Texas_Hold'em_Poker

5. **Roboflow Computer Vision Platform**
https://roboflow.com